

BUILD-SUPABASE.md

Partie C.5 — Contraintes PostgreSQL

Version : V1

Statut : Référence officielle des contraintes PostgreSQL

1. Objectif

Cette section définit les règles officielles d'utilisation des contraintes PostgreSQL dans Vitala.

Les contraintes garantissent :

- l'intégrité des données ;
- la cohérence entre les domaines ;
- la qualité des informations enregistrées ;
- la prévention des erreurs de saisie et de développement.

Les contraintes constituent un mécanisme de protection obligatoire de la base de données.

2. Principes directeurs

Toutes les contraintes doivent respecter les principes suivants :

- Integrity by Design
- Domain First
- Simplicité
- Cohérence
- Performance
- Documentation obligatoire

Une règle métier ne remplace jamais une contrainte technique lorsque PostgreSQL peut la faire respecter.

3. PRIMARY KEY

Chaque table doit posséder une clé primaire.

Règles :

- type `UUID` ;

- valeur unique ;
- non modifiable ;
- non nulle.

La clé primaire identifie définitivement chaque enregistrement.

4. FOREIGN KEY

Les relations entre tables doivent être protégées par des clés étrangères.

Objectifs :

- empêcher les références invalides ;
- maintenir la cohérence des relations ;
- faciliter les jointures.

Toute clé étrangère doit être documentée.

5. UNIQUE

Une contrainte `UNIQUE` est utilisée lorsqu'une valeur ne peut apparaître qu'une seule fois.

Exemples :

- identifiant public ;
- adresse e-mail ;
- code d'invitation ;
- numéro de référence.

Lorsque l'unicité dépend de plusieurs colonnes, une contrainte composite doit être utilisée.

6. NOT NULL

Toute colonne indispensable au fonctionnement du domaine doit être déclarée `NOT NULL`.

Les colonnes facultatives doivent être justifiées.

Le recours aux valeurs `NULL` ne doit jamais servir à contourner une règle métier.

7. CHECK

Les contraintes `CHECK` permettent de limiter les valeurs autorisées.

Exemples :

- score supérieur ou égal à 0 ;
- pourcentage compris entre 0 et 100 ;
- date de fin postérieure à la date de début ;
- quantité strictement positive.

Les règles simples doivent être appliquées directement par PostgreSQL.

8. DEFAULT

Les valeurs par défaut doivent être définies lorsque cela améliore la cohérence des données.

Exemples :

- date de création ;
- état initial ;
- indicateurs booléens.

Les valeurs par défaut doivent être documentées et rester cohérentes avec les règles métier.

9. Contraintes composites

Lorsque plusieurs colonnes définissent ensemble une règle d'unicité ou de validité, une contrainte composite doit être créée.

Exemple :

- un même utilisateur ne peut posséder qu'un seul profil principal.
-

10. Contraintes d'intégrité référentielle

Les suppressions et mises à jour doivent être définies explicitement :

- `RESTRICT`
- `CASCADE`
- `SET NULL`
- `SET DEFAULT`
- `NO ACTION`

Le choix dépend du comportement attendu du domaine métier.

L'utilisation de `CASCADE` doit être exceptionnelle et justifiée.

11. Contraintes métier

Les règles métier complexes doivent être implémentées dans la couche Domaine.

Les contraintes PostgreSQL sont réservées aux règles pouvant être garanties directement par la base de données.

Cette séparation préserve l'architecture Domain-Driven.

12. Performance

Les contraintes doivent être conçues de manière à garantir l'intégrité sans dégrader inutilement les performances.

Les contraintes inutiles ou redondantes sont interdites.

13. Documentation

Chaque contrainte doit être documentée avec :

- son objectif ;
 - les colonnes concernées ;
 - le domaine propriétaire ;
 - les impacts éventuels sur les performances.
-

14. Évolution

Toute modification d'une contrainte doit :

- passer par une migration ;
 - être analysée pour mesurer son impact ;
 - préserver les données existantes ;
 - être documentée.
-

15. Gestion des erreurs

Les violations de contraintes doivent produire des erreurs explicites.

Les APIs traduisent ces erreurs en messages compréhensibles pour les clients, sans exposer de détails techniques internes.

16. Critères de conformité

Les contraintes sont conformes lorsque :

- chaque table possède une clé primaire ;
 - les relations utilisent des clés étrangères ;
 - les champs obligatoires sont protégés par `NOT NULL` ;
 - les règles simples sont appliquées via `CHECK` ou `UNIQUE` lorsque pertinent ;
 - les contraintes sont documentées ;
 - les migrations sont versionnées.
-

17. Conclusion

Les contraintes PostgreSQL constituent la garantie fondamentale de l'intégrité des données de Vitala.

Elles complètent les validations réalisées par les APIs et les domaines métier afin d'assurer une plateforme fiable, cohérente et durable.